

DIPLOMARBEIT

State of the Art Rendering

im Projekt GPU Cluster Management System

Ausgeführt im Schuljahr 2026/27 von:

Martin Hochstrasser, 5AHIF-07
Christoph Trappl, 5AHIF-23

Betreuer/Betreuerin:

Dipl.-Ing. Christoph Schreiber

St.Pölten, am 14. September 2026

Eidesstattliche Erklärung

Ich erkläre an Eides statt, dass ich die vorliegende Diplomarbeit selbständig und ohne fremde Hilfe verfasst, andere als die angegebenen Quellen und Hilfsmittel nicht benutzt und die den benutzten Quellen wörtlich und inhaltlich entnommenen Stellen als solche erkenntlich gemacht habe.

Martin Hochstrasser

Christoph Trappl

St.Pölten, am 20.04.2015

Diplomandenvorstellung



Max MUSTERMANN

Geburtsdaten:
06.02.1996 in Musterort

Wohnhaft in:
Musterstraße 13/1
3100 Musterstadt

Werdegang:
2010 - 2015:
HTBLuVA St.Pölten, Abteilung für Informatik
2006 - 2010:
Bundesrealgymnasium Wieselburg a. d. Erlauf

Kontakt:
max.mustermann@gmx.at

Danksagungen

Danke

Zusammenfassung

Die Java Schuladministrationssoftware - jSAS:

Ziel des Projektes jSAS ist die Portierung der bestehenden, auf Servlets basierenden, SASII Weboberfläche zu einer benutzerfreundlichen Anwendung mit ergonomischer Oberfläche.

Um eine Performancesteigerung zur Vorversion zu erreichen und das Rechtemanagement besser auf die Anforderungen abzustimmen, wird anstatt einer PostgreSQL™ Datenbank die neueste Version der SAP™ Datenbank benutzt. Weiters soll die Rechtevergabe dynamisch und durch das Programm administrierbar sein.

Die Verteilung des Programms und auch das automatische Einspielen von Updates und Patches soll mit Java WebStart™ realisiert sein.

Übersicht über die Diplomarbeit:

Diese Diplomarbeit stellt die Neuerungen der Version 1.5 der Java Entwicklungsumgebung sowie der Java Laufzeitumgebung vor. Besondere Berücksichtigung finden die Einführung von generischen Konstrukten und Verbesserungen in der Syntax.

Die Ergebnisse werden speziell auf ihre Einsetzbarkeit im Projekt JSAS untersucht.

Abstract

The Java School Administration Software - jSAS:

The main goal of the project jSAS is the transformation of the existing SASII web portal, based on servlets, to an ergonomically and user friendly client application.

To achieve an improvement of performance and for a better adjustment of rights management to the customers needs the new version uses the latest release of a SAP™ database instead of the one from PostgreSQL™ . The project offers a management tool which enables the administrator to adjust user rights dynamically.

Distribution, automatic updates and patches of the jSAS software are provided over the Java WebStart™ technology.

Overview on the diploma thesis:

This diploma thesis deals with the innovations of the Java runtime and development environment release 1.5. Special consideration is placed on the introduction of generic constructs and improvements on the language syntax.

The results are especially investigated on their usefulness in the jSAS project.

Inhaltsverzeichnis

Vorwort	i
Diplomandenvorstellung	iii
Danksagungen	v
Zusammenfassung	vii
Abstract	ix
Inhaltsverzeichnis	xi
1 Übersicht	1
1.1 Der Java Community Process™ (JCP)	1
1.1.1 Aufgaben und Ziele des JCP	1
1.1.2 Java Specification Request (JSR)	2
1.2 Die Vorgänger Java 1.4.1 und Java 1.4.2	3
1.2.1 Java 1.4.1 "Hopper"	3
1.2.2 Java 1.4.2 "Mantis"	3
1.3 Neuerungen in Java 1.5 "Tiger"	4
2 Generische Datentypen	5
2.1 Definition	5

2.1.1	Was sind Generische Datentypen?	5
2.1.2	Umsetzung generische Datentypen	7
2.2	Deklaration	7
2.2.1	Klassenschablonen	7
	Einfügen von Grafiken	9
2.2.2	Richtiges zitieren	11
2.2.3	Einbinden von PDF - Seiten	11

Anhang	14
---------------	-----------

	Abbildungsverzeichnis	14
	Tabellenverzeichnis	17
	Verzeichnis der Listings	19
	Literaturverzeichnis	21

Kapitel 1

Übersicht

1.1 Der Java Community Process™ (JCP)

Seit 1998 existiert eine Unterabteilung von Sun Microsystems™ welche sich mit der Weiterentwicklung der Java Technologie beschäftigte [Re:Book02]. Diese Abteilung wurde in den Jahren 2000 und 2002 in Zusammenarbeit mit großen Industriefirmen öffentlich zugänglich gemacht und als so genannter "Java Community Process" eingerichtet welcher von einem Executive Committee geleitet wird.

1.1.1 Aufgaben und Ziele des JCP

Die Aufgabe ¹des JCP ist es neue APIs² für die "Evolution" der Programmiersprache Java zu entwickeln, zu normieren und zu verbessern. Diese Aufgabe wird in allen Bereichen von Java (J2ME³, J2SE⁴, J2EE⁵ und den zugehörigen Technologien (XML, Web Services etc.) mit Hilfe von Experten aus der Industrie wahrgenommen.

Hierunter finden sich Firmen wie Siemens, Nokia, Sony-Ericsson, Palm, Texas Instruments, etc. für die Java 2 Micro Edition und Apache, Apple, Borland, Cisco, HP, IBM, Oracle, SAP etc. für die Standard und Enterprise Edition aber auch einzelne Personen welche die Java Entwicklung vorantreiben möchten.

¹hfjhfsjhfsdj

²Application Programming Interfaces

³Java 2 Micro Edition

⁴Java 2 Standard Edition

⁵Java 2 Enterprise Edition

Das Ziel des JCP ist ein binärer Softwarestandard, welcher die *"write once, run anywhere"* Devise von Java für die Entwickler garantieren soll.

1.1.2 Java Specification Request (JSR)

Ein JSR durchläuft vier verschiedene Phasen und erzeugt eine Java API Spezifikation, eine Referenzimplementierung und eine Kompatibilitätstest-Umgebung.

Die vier Phasen eines JSRs sind:

1. Initiation:

Ein JSR wird von Mitgliedern der Community eingebracht und durch das Executive Committee des JCP genehmigt.

2. Community Draft:

Eine Expertengruppe wird zusammengestellt, die einen Entwurf erarbeitet, welcher der Community vorgelegt wird. Nachdem das Feedback der Community in den Entwurf eingearbeitet wurde wird dieser dem Executive Committee übergeben und nach Bestätigung in die nächste Phase übergeben.

3. Public Draft:

Der bisherige Entwurf wird allgemein zugänglich gemacht und mit Hilfe der öffentlichen Meinung einer weiteren Revision unterzogen. Weiters werden die Referenzimplementierung und die Umgebung für Kompatibilitätstests in dieser Phase erstellt.

4. Maintenance:

Der fertige Entwurf wird in die Java Umgebung eingearbeitet und durch weitere Requests und Revisionen aktualisiert. Inwiefern solche Aktualisierungen in den bestehenden Entwurf eingearbeitet werden, wird durch das Executive Committee festgelegt.

Dem JCP gelang es mit über 650 Mitgliedern und über 200 JSRs die größte öffentliche Plattform für die Weiterentwicklung einer Programmiersprache zu werden. Sie ermöglicht eine dynamische und fortlaufende Anpassung der Sprache an die Anforderungen der Programmierer und der Industrie.

Weiters unterzieht sich der JCP selbst regelmäßig einer Verbesserung durch sein eigenes JSR-Konzept, um auch hier die bestmögliche Ausnutzung der Ressourcen und Möglichkeiten zu erreichen.⁶

⁶Aktuelle Version des JCP: 2.5

1.2 Die Vorgänger Java 1.4.1 und Java 1.4.2

Da die J2SE Plattform bereits weit verbreitet ist und sowohl auf Servern als auch auf Clients für die unterschiedlichsten Anwendungsgebiete benutzt wird, konzentrierte sich Sun Microsystems in den letzten beiden Releases der J2SE Plattform vor allem auf Bugfixes und Verbesserungen der Performance.

1.2.1 Java 1.4.1 "Hopper"

- Über 2.000 Bugfixes
- Zwei neue Garbage Collector Implementationen
- Reduzierte Downloadgröße

1.2.2 Java 1.4.2 "Mantis"

- Nochmals über 2.000 Bugfixes
- 15-30% niedrigere Startzeiten⁷
- Zwei neue Swing Look-and-Feels (Windows XP™ und Linux GTK)
- Neuer Windows Installer und nochmalige Reduzierung der Downloadgröße
- ActiveX™ Brücke

Eine automatische Updatefunktion für die JRE™⁸ wurde auf ein zukünftiges Release verschoben, da die nötige Testphase nicht abgeschlossen werden konnte.⁹

Die Java Grafikbibliothek Swing erfuhr bis zur Version 1.4.2 insgesamt eine 50-prozentige Performancesteigerung gegenüber der Version 1.3.1.

⁷Vor allem auf Linux/x86 Systemen

⁸Java Runtime Environment

⁹Sie wurde in einem späteren Release der Version 1.4.2 eingebracht.

1.3 Neuerungen in Java 1.5 "Tiger"

Die neue Java Version 1.5 wurde komplett durch den JCP entwickelt und enthält 15 JSRs für neue Features und Pakete:

XXXXXXXXXXXXXXXXXXXXX	XXXXXXXXXXXXXXXXXXXXX
JSR 003: JMX™ Management API	JSR 166: Concurrency Utilities
JSR 013: Decimal Arithmetic	JSR 174: JVM™ Monitoring
JSR 014: Generic Types	JSR 175: Metadata
JSR 028: SASL	JSR 199: Compiler APIs
JSR 114: JDBC™ API Rowsets	JSR 200: Pack Transfer Format
JSR 133: New Memory Model	JSR 201: Four Language Updates
JSR 163: Profiling API	JSR 204: Unicode Surrogates
	JSR 206: JAXP 1.3

Tabelle 1.1: JSRs enthalten in Java 1.5

Weitere Verbesserungen sollen wie immer in den Bereichen

- Performance (HotSpot tuning, NIO API¹⁰, Startzeit, etc.)
- JVM™¹¹ Management und Überwachung
- Grafik (Look-and-Feels, OpenGL, etc.)
- XML (XML 1.1, XML Schemas, etc.)
- Web Services
- JDBC™¹²

stattfinden.

Die wichtigsten Änderungen der neuen Java Version werden in den nachfolgenden Kapiteln dieser Arbeit näher betrachtet.

¹⁰New Input/Output API

¹¹Java Virtual Machine

¹²Java Database Connection

Kapitel 2

Generische Datentypen

2.1 Definition

2.1.1 Was sind Generische Datentypen?

Die Einführung generischer Datentypen, so genannter "*Generics*", stellt wohl die größte Änderung für Programmierer seit Einführung der J2SE¹ dar.

Die grundlegende Funktion generischer Datentypen lässt sich am besten mittels eines Beispiels erklären: Angenommen man würde eine einfache Klasse implementieren wollen, welche einen String kapselt. Diese Klasse könnte in etwa so aussehen:

```
1
2 // Test Umlaute in Listings: öäüÖÄÜß
3
4 public class StringWrapper
5 {
6     private String value;
7
8     public StringWrapper() {}
9
10    public StringWrapper(String value) {
11        this.value=value;
12    }
13
14    public void setValue(String value) {
15        this.value=value;
```

¹Java 2 Standard Edition

```
16 }
17
18 public String getValue() {
19     return this.value;
20 }
21 }
```

Listing 2.1: Kapselung eines Strings

Würde man nun dieselbe Klasse für den Typ Integer implementieren wollen, so würde sie so aussehen:

```
1 public class IntegerWrapper
2 {
3     private Integer value;
4
5     public IntegerWrapper() {}
6
7     public IntegerWrapper(Integer value) {
8         this.value=value;
9     }
10
11     public void setValue(Integer value) {
12         this.value=value;
13     }
14
15     public Integer getValue() {
16         return this.value;
17     }
18 }
```

Listing 2.2: Kapselung eines Integers

Wie man sieht braucht man für die Kapselung eines anderen Datentyps eine neue Klasse. Diese beinhaltet dieselbe Logik wie die Originalklasse, allerdings mit dem neuen Datentyp in den entsprechenden Methodendefinitionen.

Die angewandte Logik und die vorkommenden Algorithmen sind also *generisch*².

Um das Erstellen mehrerer Klassen für verschiedene Datentypen umgehen zu können wird in Java 1.5 das Konzept der Generics eingeführt. Sie erlauben es bestimmte Datentypen erst im Anwendungsfall festzulegen und ermöglichen die Definition in nur einer generischen Klasse.

²vom Datentyp unabhängig

2.1.2 Umsetzung generische Datentypen

Es gibt zwei Realisierungsmöglichkeiten von generischen Datentypen:

- Heterogene Variante: Für jeden Typ (z.B. String, Integer) wird individueller Code erzeugt, also z.B. zwei Klassen.
- Homogene Übersetzung: Für jede parametrisierte Klasse wird eine Klasse erzeugt, die statt des generischen Typs Object verwendet. Für einen konkreten Typ werden Typcasts in die Anweisungen eingebaut.

Beide Realisierungsmöglichkeiten werden durch die Compiler oder Interpreter der entsprechenden Sprachen durchgeführt und bleiben dem Programmierer einer generischen Klasse verborgen.

Java benutzt die homogene Übersetzung. Die generischen Klassen lassen sich außerdem weiterhin unter dem Typ Object benutzen.

2.2 Deklaration

2.2.1 Klassenschablonen

Um eine generische Klasse aus den oben vorgestellten Klassen zu erzeugen, genügt es alle Vorkommnisse des entsprechenden Datentyps durch einen speziellen Bezeichner zu ersetzen. Dieser muss in der Klassendefinition in spitzen Klammern angegeben werden und wird meist T (für Typ) genannt.

```
1 public class GenericWrapper<T> {  
2     private T value;  
3  
4     public GenericWrapper() {}  
5  
6     public GenericWrapper(T value) {  
7         this.value=value;  
8     }  
9  
10    public void setValue(T value) {  
11        this.value=value;  
12    }  
13  
14    public T getValue() {  
15        return this.value;  
16    }  
17 }
```

```

16 }
17 }

```

Listing 2.3: GenericWrapper.java

```

1 public class GenericWrapper<T>
2 {
3     private T value;
4
5     public GenericWrapper() {}
6
7     public GenericWrapper(T value) {
8         this.value=value;
9     }
10
11    public void setValue(T value) {
12        this.value=value;
13    }
14
15    public T getValue() {
16        return this.value;
17    }
18 }

```

Listing 2.4: Kapselung eines generischen Datentyps

Diese Art eine generische Klasse zu definieren nennt sich Klassenschablone. Um die Klasse nun für bestimmte Datentypen nutzen zu können muss sie für den entsprechenden Typ parametrisiert werden:

```

1 GenericWrapper<String>  stringWrapper  = new GenericWrapper<String> ();
2 GenericWrapper<Integer> integerWrapper = new GenericWrapper<Integer> ();
3 GenericWrapper<Double>  doubleWrapper  = new GenericWrapper<Double> ();

```

Listing 2.5: Benutzen einer generischen Klasse

Der Vorteil dieser Parametrisierung liegt in der Möglichkeit schon während des Kompiliervorgangs prüfen zu können ob eine Zuweisung bzw. ein entsprechender Aufruf korrekt ist. Außerdem wird es dem Programmierer ermöglicht Daten ohne Typcasts zu verwenden, da der Inhalt der parametrisierten Klasse von vorne herein bekannt ist:

```

1 //Folgendes Statement produziert einen Kompilierfehler:
2 GenericWrapper<String> stringWrapper =
3     new GenericWrapper<Integer> ();
4
5 //Dieser Methodenaufruf ebenfalls:
6 doubleWrapper.setValue("Test");
7
8 //Folgende Aufrufe sind hingegen erfolgreich:
9 integerWrapper.setValue(new Integer(10));

```

```
10 stringWrapper.setValue("Erfolgreich!");  
11  
12 //Das nachfolgende Statement kommt ohne Typecast aus:  
13 int t=integerWrapper.getValue().intValue()+200;
```

Listing 2.6: Benutzen einer generischen Klasse

Würde man obige Zeile ohne generische Datentypen programmieren so würde sich, wie bisher, folgender Code ergeben:

```
1 IntegerWrapper integerWrapper = new IntegerWrapper(new Integer(10));  
2  
3 int t=((Integer) integerWrapper.getValue()).intValue()+200;
```

Listing 2.7: Benutzen der nicht generischen Integer Klasse

Angenommen man würde dieselbe Wrapper-Klasse ohne Generics für die Klasse `Number` implementieren, so könnte man in ihr wie oben `Integers` speichern. Es wäre aber auch möglich `numberWrapper.setValue(new Double(10.5))` auszuführen, da sowohl `Integer` als auch `Double` von `Number` erben.

Hier würde das Programm aber zur Laufzeit in Zeile 3 eine `ClassCastException` werfen, da eine `Double` - Instanz nicht zu einem `Integer` gecastet werden kann.

Befindet sich dieser Cast nun in selten genutzten Programmteilen und innerhalb eines komplexeren Anwendungsfalles, in dem z.B. die Werte der Klasse in unterschiedlichen Methoden geschrieben und gelesen werden, so kann es vorkommen, dass dieser Fehler von den Programmieren übersehen wird.

Dies wäre z.B. der Fall, wenn nur in bestimmten Programmsituationen ein `Double` in die Klasse geschrieben würde, aber immer ein `Integer` für die Berechnung der Zeile 3 verwendet werden würde.

Durch den Einsatz generischer Datentypen, Klassenschablonen und parametrisierter Klassen lassen sich diese Fehler bereits zum Zeitpunkt der Kompilierung feststellen und vermeiden. Entwickler können so wertvolle Zeit sparen und das Auftreten von *Runtime Exceptions* wird vermindert.

Einfügen von Grafiken

Ein Bild, das nicht zum Thema passt.



Abbildung 2.1: Norwegen - Wegweiser



Abbildung 2.2: Neues HTL - Logo

2.2.2 Richtiges zitieren

Dies ist ein wörtliches Zitat:

Textverarbeitung durch einen Rechner mit dem Ergebnis des Ausdrucks in Buchqualität ist durch die Entwicklung geeigneter Programme in den letzten Jahren möglich geworden.[Re:Book02, Kopka, 2003, Seite 3]

Bezug auf eine bestimmte Quelle in einem Buch:

Dieser Sachverhalt wird in [Re:Book02, Deming Klaus, 2002, Seite 17ff] genauer beschrieben.

Eine gute Einführung zu $\text{\LaTeX}$ findet man unter [Re:Web01, LatexTutorial der Uni Hamburg]

2.2.3 Einbinden von PDF - Seiten

Hier wird (auf der nächsten Seite) ein externes PDF-Dokument eingefügt:

Hausübung 03 5XHIF 26.02.2015

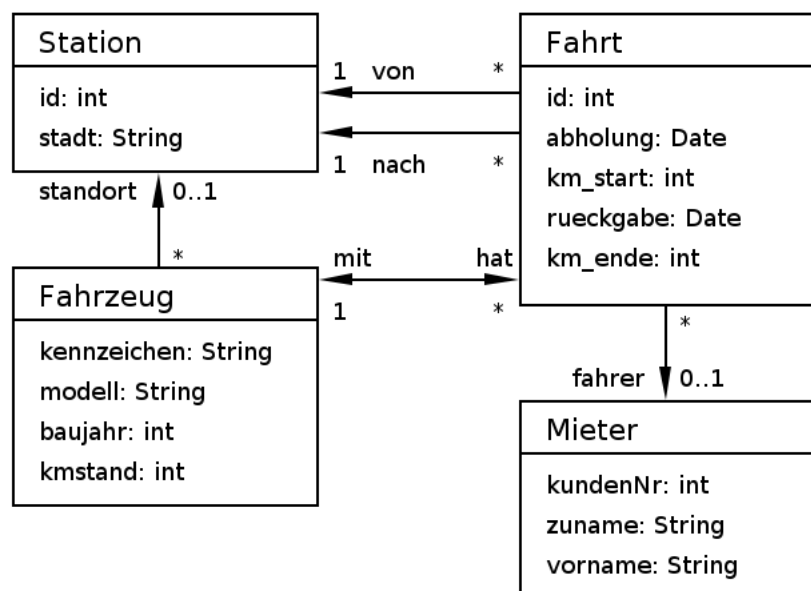
Problemstellung

Die Deluxe-Car4Rent-GesmbH möchte eine Webapplikation entwickeln, in der die Daten zu den einzelnen Stationen, den Mietern, den Fahrzeugen und deren Fahrten erfasst und dargestellt werden können.

Ihre Aufgabe ist es, unter Berücksichtigung des gegebenen UML-Diagramms und der definierten Geschäftsregeln einen Prototypen nach folgenden Vorgaben zu entwickeln:

1. Die Webapplikation ist mit Hilfe von Java EE unter Verwendung von JSF 2.0 zu entwickeln. Dabei ist die Facelet-Technologie zu verwenden, als externe Komponentenbibliothek werden Primefaces empfohlen.
2. Datenbankzugriffe sind mit Hibernate zu realisieren, als DBMS ist PostgreSQL zu verwenden.

UML-Diagramm



Geschäftsregeln

- Ein Mieter kann ein Fahrzeug an einer Vermietstation abholen und an einer anderen wieder abgeben (repräsentiert durch die Assoziationen von und nach).
- Einer noch nicht abgeschlossenen Fahrt ist noch keine Station über die nach-Beziehung zugeordnet.
- Ein Fahrzeug, das gerade unterwegs ist (also einer noch nicht abgeschlossenen Fahrt zugeordnet ist), hat keinen Standort an einer der Vermietstationen.
- Jeder Fahrt muss ein Mieter zugeordnet sein.
- Fahrten stellen eine Historisierung von Vorgängen dar, daher dürfen die initial getroffenen Zuordnungen zu Abfahrtsort, Mieter und Fahrzeug nicht nachträglich geändert werden.

Aufgaben

Aufgabe 1 - Geschäftsmodell und Testdaten

Erstellen Sie ein geeignetes Entitymodell und entwickeln Sie zugehörige Geschäftsklassen mit Hibernate - Annotations. In der Entity Fahrzeug ist das Kennzeichen als ID zu verwenden, Mieter erhalten die in der gegebenen XML-Datei definierten ID's. Alle anderen Primarykeys sind automatisch zu erzeugen.

Zusätzlich zu den durch das gegebene UML-Diagramm beschriebenen Klassen sind in der Datenbank Logindaten (Username und Passwort) zu wenigstens 3 Mitarbeitern zu erfassen. Für diesen Prototyp darf das Passwort im Klartext in der Datenbank gespeichert sein.

Schreiben Sie eine Konsole Applikation, welche aus dem erstellten Entitymodell mit Hilfe eines Schemaexportes eine Datenbank erzeugt und die jede Tabelle mit wenigstens 3 Datensätzen füllt. Bei den Fahrten ist auch eine nicht abgeschlossene Fahrt in den Testdaten vorzusehen.

Aufgabe 2 - Webinterface

Entwickeln Sie mit Hilfe von JavaServerFaces und Primefaces eine auf Facelets basierende Webapplikation mit folgender Funktionalität:

- **Loginseite**
Mit Hilfe dieses Formulars kann sich ein Mitarbeiter anmelden. Stellen Sie über ein Cookie sicher, dass der Loginname vorausgefüllt ist, wenn ein User innerhalb von 30

Abbildungsverzeichnis

2.1	Norwegen - Wegweiser	10
2.2	Neues HTL - Logo	10

Tabellenverzeichnis

1.1 JSRs enthalten in Java 1.5	4
--	---

Listings

2.1	Kapselung eines Strings	5
2.2	Kapselung eines Integers	6
2.3	<code>GenericWrapper.java</code>	7
2.4	Kapselung eines generischen Datentyps	8
2.5	Benutzen einer generischen Klasse	8
2.6	Benutzen einer generischen Klasse	8
2.7	Benutzen der nicht generischen Integer Klasse	9

Literaturverzeichnis

[Re:Book02] Helmut Kopka: *Latex Band 1, Einführung*
Addison-Wesley, 2000
ISBN: 3-8273-7038-8

[Re:Book02] Demmig, Thomas:
jetzt lerne ich Latex 2
Markt+Technik, 2004
ISBN 3-8272-6517-7

[Re:Web01] <http://www.meta-x.de/faq/LaTeX-Einfuehrung.html>
Latex-Einführung
28.September 2012

[Re:Web02] http://docs.oracle.com/cd/E12839_01/core.1111/e10043/introjps.htm
Oracle Security Guide über das Java Sicherheits Model
13.11.2014

